

P0543R0: Saturation arithmetic

Introduction

Arithmetic operations on unsigned integers in C and C++ wrap on overflow or underflow (3.9.1 [basic.fundamental] p4):

Unsigned integers shall obey the laws of arithmetic modulo 2^n where n is the number of bits in the value representation of that particular size of integer.

Signed integer operations have undefined behavior on overflow or underflow (5 [expr] p4):

If during the evaluation of an expression, the result is not mathematically defined or not in the range of representable values for its type, the behavior is undefined.

In order to implement some algorithms, the use of saturation arithmetic is necessary, where an operation causing overflow or underflow instead returns the smallest or largest representable number. For example, when determining the color of a pixel, it would not make sense that brightening a white pixel suddenly turns it black or dark-grey.

This paper proposes to add simple free functions for basic saturating operations on all signed and unsigned integer types. Further, a `saturate_cast<T>` is provided that can convert from any of those types to any other, saturating the value as needed.

A previous proposal was in "N3864: A constexpr bitwise operations library for C++" by Matthew Fioravante, but only for addition and subtraction.

It is expected that the functions provided with this proposal will be, at some later time, overloaded for `std::datapar`, the nascent SIMD data type (see P0214R2 "Data-Parallel Vector Types & Operations" by Matthias Kretz).

Examples

The following examples assume `CHAR_BIT == 8`.

```
int x1 = satadd(3, 4);           // ok, yields 7
int x2 = satsub(INT_MIN, 1);     // ok, yields INT_MIN
unsigned char x3 = satadd(255, 4); // compiles, but yields 3
unsigned char x4 = satadd<unsigned char>(255, 4); // ok, yields 255
unsigned char x5 = satadd(252, x3); // error: inconsistent deductions for T
unsigned char x6 = satadd<unsigned char>(251, x1); // ok, yields 255; might yield an int -> unsigned char conversion warning for x1
```

Design considerations

All of addition, subtraction, multiplication, and division are provided.

The operations are not defined on the integral types `bool`, `char`, `wchar_t`, `char16_t`, and `char32_t`, as these are not intended for arithmetic.

Unlike the built-in arithmetic operators on integers, this proposal expressly does not apply integral promotions to the arguments, since that would be besides the point for saturation arithmetic.

The situation for template argument deduction presented by these functions is the same as for `std::min` or `std::max`: If two arguments of different type are passed, the call fails to compile.

Instead of free functions, it is conceivable to provide an integer-like class template with the arithmetic operators suitably overloaded. This would, however, make it impossible to adopt this proposal for C, and seems slightly over-engineered for a rather simple facility.

The header `<cmath>` contains mostly (except for `abs`) floating-point functions, so integer-only arithmetic functions do not seem to fit. The header `<cstdlib>` does contain `abs` and `div` functions for integers, but its contents is mostly defined by the related C header `<stdlib.h>`, therefore I suggest to create a new header.

Prior art

A lot of SIMD instruction sets contain CPU instructions for saturation arithmetic on SIMD vectors, including SSE2 for x86 and NEON for ARM.

A branch-free implementation for scalars is available here: https://locklessinc.com/articles/sat_arithmetic/.

Wording

Add the header `<saturation>` to the table in 17.5.1.2 [headers].

Append a new subsection to clause 26 [numerics] with the following content:

26.10 Saturation arithmetic [numerics.sat]

26.10.1 Header `<saturation>` synopsis [saturation.syn]

```
namespace std {
    template<class T>
        T satadd(T x, T y) noexcept;
    template<class T>
        T satsub(T x, T y) noexcept;
    template<class T>
        T satmul(T x, T y) noexcept;
    template<class T>
        T satdiv(T x, T y);
    template<class T, class U>
        T saturate_cast(U x);
}
```

26.10.2 Arithmetic functions [numerics.sat.func]

[Note: In the following descriptions, an arithmetic operation is performed as a mathematical operation with infinite range and then it is determined whether the mathematical result fits into the result type.]

```
template<class T>
    T satadd(T x, T y) noexcept;
```

Returns: If $x + y$ is representable as a value of type T , $x + y$, otherwise either the largest or smallest representable value of type T , whichever is closer to the value of $x + y$.

Remarks: Participates in overload resolution only if T is a signed or unsigned integer type (3.9.1 [basic.fundamental]).

```
template<class T>
    T satsub(T x, T y) noexcept;
```

Returns: If $x - y$ is representable as a value of type T , $x - y$, otherwise either the largest or smallest representable value of type T , whichever is closer to the value of $x - y$.

Remarks: Participates in overload resolution only if T is a signed or unsigned integer type (3.9.1 [basic.fundamental]).

```
template<class T>
    T satmul(T x, T y) noexcept;
```

Returns: If $x * y$ is representable as a value of type T , $x * y$, otherwise either the largest or smallest representable value of type T , whichever is closer to the value of $x * y$.

Remarks: Participates in overload resolution only if T is a signed or unsigned integer type (3.9.1 [basic.fundamental]).

```
template<class T>
    T satdiv(T x, T y);
```

Let q be x / y , with any fractional part discarded.

Returns: If q is representable as a value of type T , q , otherwise either the largest or smallest representable value of type T , whichever is closer to the value of q .

Throws: Nothing.

Remarks: The behavior is undefined if y is zero. Participates in overload resolution only if T is a signed or unsigned integer type (3.9.1 [basic.fundamental]).

26.10.3 Casting [numerics.sat.cast]

```
template<class T, class U>
    T saturate_cast(U x);
```

Returns: If x is representable as a value of type T , x , otherwise either the largest or smallest representable value of

type T, whichever is closer to the value of x.

Remarks: Participates in overload resolution only if T and U are signed or unsigned integer types (3.9.1 [basic.fundamental]).

References

- ISO/IEC JTC1 SC22 WG21 [N3864](#): "A constexpr bitwise operations library for C++" by Matthew Fioravante
- ISO/IEC JTC1 SC22 WG21 [P0214R2](#): "Data-Parallel Vector Types & Operations" by Matthias Kretz
- Wikipedia: [Saturation arithmetic](#)
- [ARM Compiler User Guide](#) [large PDF], section 13.89 "QSUB" instruction reference
- [Intel x86 Instruction Set Reference](#) [large PDF], PSUBUSB instruction